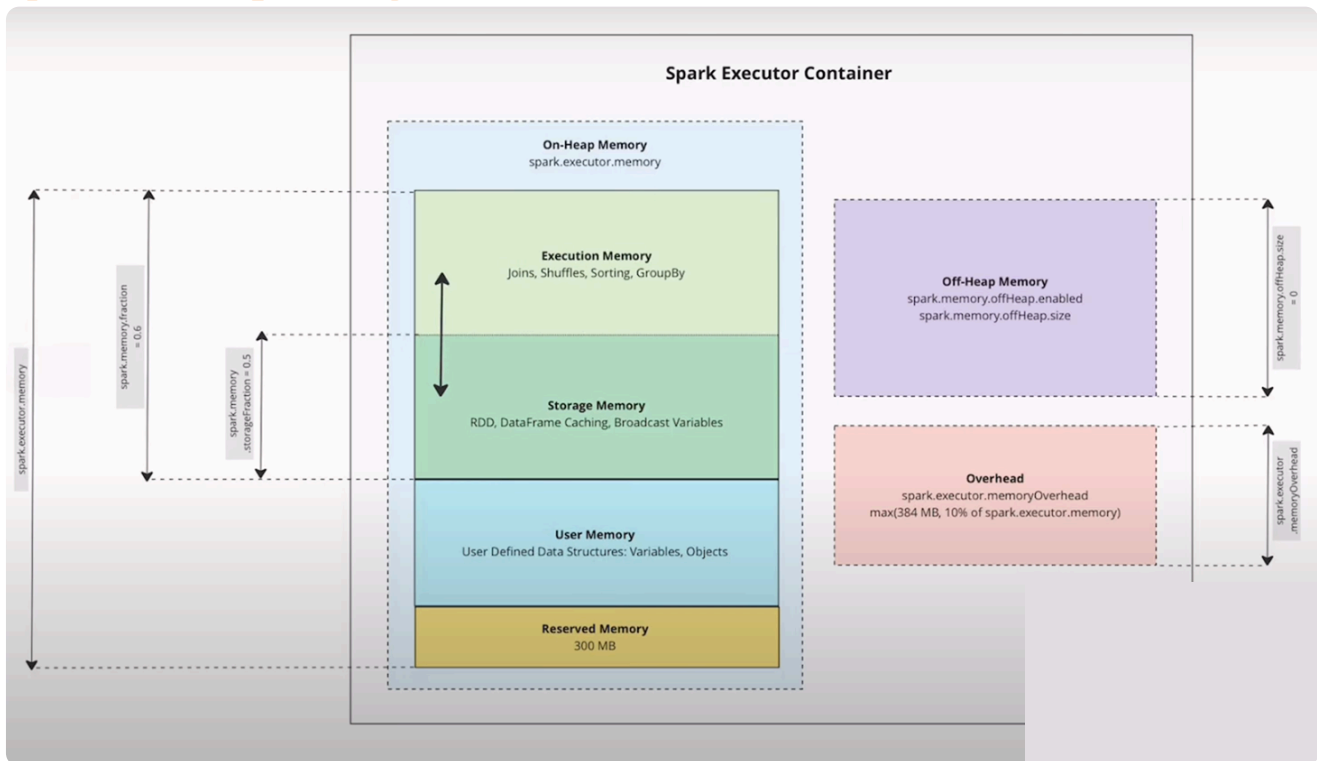


13 Spark Memory Management

Spark Memory Management



Overview

Understanding Spark's memory management is **crucial** for:

- Solving optimization problems
- Developing solid grasp of Spark internals
- Understanding memory allocation patterns
- Debugging performance issues
- Optimizing resource utilization

Key Insight: Memory management directly impacts Spark application performance, stability, and resource efficiency.

Core Memory Management Areas

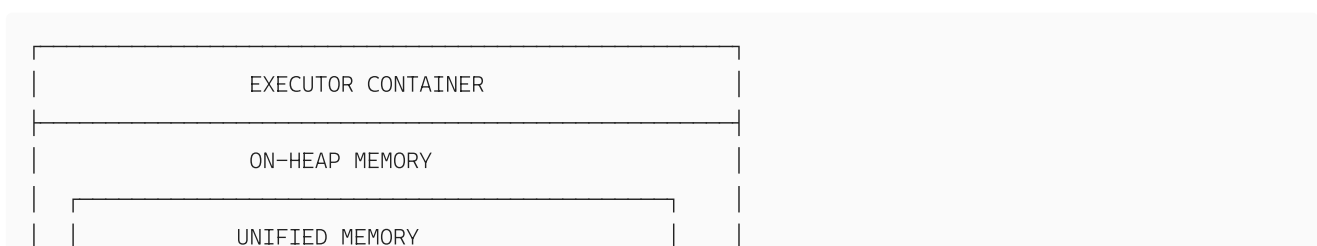
Spark's memory management encompasses three fundamental areas:

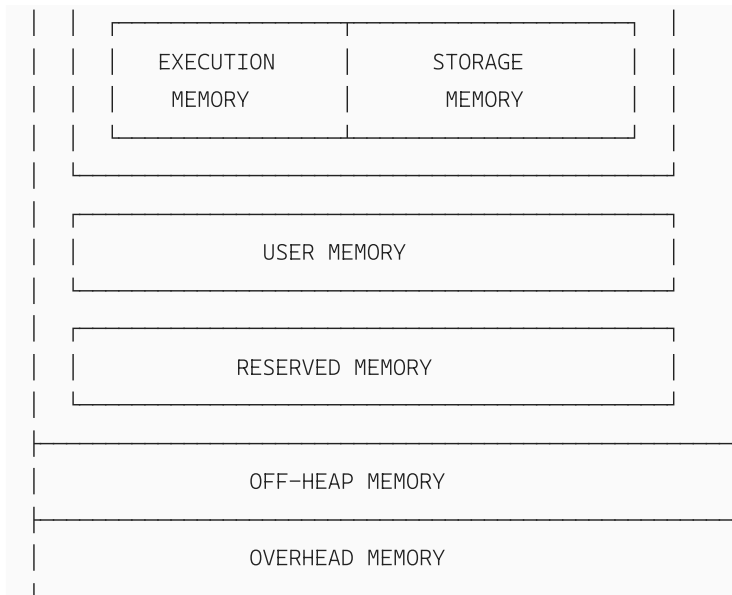
1. **Executor Memory Management** - Primary memory allocation within executors
2. **Unified Memory** - Dynamic memory sharing between execution and storage
3. **Off-heap Memory** - Alternative memory management outside JVM

Executor Memory Management

Memory Architecture Overview

A Spark executor container consists of three major memory components:





On-Heap Memory Components

- Most of the things happen here.
- Managed by the JVM.

Unified Memory -> Exec + Storage. Called Unified because of the dynamic memory management of the spark.

1. Execution Memory

- **Purpose:** Space for computationally intensive operations
- **Operations:**
 - Joins (hash joins, sort-merge joins)
 - Shuffles (data redistribution)
 - Sorts (ordering operations)
 - Aggregations (groupBy, reduceByKey)
- **Characteristics:** High priority in memory allocation
- **Impact:** Directly affects query performance

2. Storage Memory

- **Purpose:** Data persistence and broadcasting
- **Uses:**
 - RDD/DataFrame caching (`cache()` , `persist()`)
 - Broadcast variables storage
 - Temporary result storage
- **Eviction:** Uses LRU (Least Recently Used) algorithm
- **Trade-offs:** Balances memory usage vs. recomputation costs

3. User Memory

- **Purpose:** Application-specific data structures
- **Contains:**
 - User-defined variables
 - Collections (Sets, Maps, Lists)
 - User-Defined Functions (UDFs)
 - Application state
- **Management:** Handled by application code
- **Size:** Typically 40% of executor memory

4. Reserved Memory

- **Purpose:** Spark internal operations

- **Size:** Fixed 300 MB
- **Contains:**
 - Spark internal objects
 - System metadata
 - Framework overhead
- **Non-configurable:** Cannot be adjusted by users

Memory Allocation Calculations

Configuration Parameters

Parameter	Default Value	Description
<code>spark.executor.memory</code>	1g	Total executor memory
<code>spark.memory.fraction</code>	0.6	Unified memory fraction
<code>spark.memory.storageFraction</code>	0.5	Storage fraction within unified memory
<code>spark.memory.offHeap.enabled</code>	false	Enable off-heap memory
<code>spark.memory.offHeap.size</code>	0	Off-heap memory size

Example Calculation (10 GB Executor)

Given: `spark.executor.memory` = 10 GB

1. Unified Memory (Execution + Storage):

$$= \text{spark.memory.fraction} \times \text{spark.executor.memory}$$

$$= 0.6 \times 10 \text{ GB} = 6 \text{ GB}$$
2. Storage Memory (within Unified):

$$= \text{spark.memory.storageFraction} \times \text{Unified Memory}$$

$$= 0.5 \times 6 \text{ GB} = 3 \text{ GB}$$
3. Execution Memory (within Unified):

$$= \text{Unified Memory} - \text{Storage Memory}$$

$$= 6 \text{ GB} - 3 \text{ GB} = 3 \text{ GB}$$
4. User Memory:

$$= (\text{spark.executor.memory} - \text{Unified Memory}) - \text{Reserved Memory}$$

$$= (10 \text{ GB} - 6 \text{ GB}) - 0.3 \text{ GB} = 3.7 \text{ GB}$$
5. Reserved Memory:

$$= \text{Fixed 300 MB} = 0.3 \text{ GB}$$

Overhead Memory

- **Purpose:** Internal system-level operations
- **Calculation:** $\max(384 \text{ MB}, 10\% \text{ of } \text{spark.executor.memory})$
- **Example:** $\max(384 \text{ MB}, 10\% \text{ of } 10 \text{ GB}) = \max(384 \text{ MB}, 1 \text{ GB}) = 1 \text{ GB}$
- **Separate:** Independent from executor memory allocation
- **Uses:**
 - JVM overhead
 - Thread management
 - Network buffers
 - Operating system operations

Total Memory Request

When requesting resources from cluster manager:

```
Total Memory Request = Executor Memory + Overhead Memory + Off-heap Memory
```

Example: 10 GB + 1 GB + 0 GB = 11 GB

Unified Memory

Conceptual Framework

Unified Memory = Execution Memory + Storage Memory

The "unified" designation reflects Spark's **dynamic memory management** capability, where memory allocation between execution and storage can be adjusted based on runtime needs.

Historical Evolution

Pre-Spark 1.6 (Static Memory Management)

- **Fixed Allocation:** Execution and storage had rigid boundaries
- **Memory Waste:** Unused memory in one area couldn't be utilized by the other
- **Performance Issues:** Frequent spills to disk due to inflexible allocation
- **Predictable:** Easier to reason about but less efficient

Post-Spark 1.6 (Dynamic Memory Management)

- **Flexible Boundaries:** Memory can be borrowed between execution and storage
- **Improved Utilization:** Better overall memory efficiency
- **Adaptive:** Responds to workload patterns
- **Complex:** Requires understanding of allocation rules

Dynamic Allocation Rules

Priority System

Execution memory has higher priority due to critical operations:

- Joins cannot be easily interrupted
- Shuffles require consistent memory allocation
- Sorting operations need contiguous memory blocks
- Aggregations benefit from stable memory allocation

Memory Borrowing Scenarios

Scenario 1: Execution Needs Memory, Storage Has Vacant Space

```
Initial State:
[Execution: 3GB Used] [Storage: 1GB Used, 2GB Vacant]
```

```
After Borrowing:
[Execution: 5GB Used] [Storage: 1GB Used]
```

- **Action:** Execution utilizes vacant storage space
- **Impact:** No eviction required
- **Performance:** Optimal scenario

Scenario 2: Execution Needs Memory, Storage Fully Occupied

```
Initial State:
[Execution: 3GB Used] [Storage: 3GB Used (Full)]

After Borrowing:
[Execution: 5GB Used] [Storage: 1GB Used (2GB Evicted)]
```

- **Action:** Storage evicts blocks using LRU algorithm
- **Impact:** Cached data may need recomputation
- **Performance:** Trade-off between execution speed and cache benefits

Scenario 3: Storage Needs Memory, Execution Occupied

```
Initial State:
[Execution: 3GB Used] [Storage: 3GB Used (Full)]

Result:
[Execution: 3GB Used] [Storage: 2GB Used (1GB Evicted)]
```

- **Action:** Storage evicts its own blocks (LRU)
- **Restriction:** Cannot evict execution blocks
- **Impact:** Cache effectiveness reduced

Optimization Strategies

Best Practices

1. **Monitor Memory Usage:** Use Spark UI to track memory utilization
2. **Selective Caching:** Cache only frequently accessed data
3. **Appropriate Storage Levels:** Choose between MEMORY_ONLY, MEMORY_AND_DISK, etc.
4. **Memory Tuning:** Adjust `spark.memory.fraction` based on workload
5. **Resource Planning:** Size executors appropriately for workload

Common Anti-patterns

- **Over-caching:** Caching too much data leads to frequent evictions
- **Under-sizing:** Insufficient executor memory causes frequent spills
- **Ignoring Execution Priority:** Assuming storage and execution have equal priority
- **Static Thinking:** Not considering dynamic memory allocation

Off-Heap Memory

Motivation and Use Cases

Garbage Collection Challenges

- **GC Pauses:** JVM garbage collection stops program execution
- **Performance Impact:** Long GC cycles significantly degrade performance
- **Memory Pressure:** Large heaps increase GC frequency and duration
- **Unpredictability:** GC timing is non-deterministic

Off-Heap Benefits

- **GC Avoidance:** Immune to JVM garbage collection
- **Consistent Performance:** Predictable memory access patterns
- **Large Memory Support:** Can handle very large datasets
- **Reduced GC Pressure:** Smaller on-heap memory reduces GC frequency

Technical Characteristics

Management Responsibility

- **Operating System Managed:** Not subject to JVM control
- **Manual Memory Management:** Developer responsible for allocation/deallocation
- **Memory Leak Risk:** Improper deallocation can cause memory leaks
- **Complexity:** Requires careful programming practices

Performance Profile

Performance Hierarchy:

1. On-heap Memory (Fastest)
2. Off-heap Memory (Intermediate)
3. Disk Storage (Slowest)

- **Slower than On-heap:** Additional indirection and OS calls
- **Faster than Disk:** Significantly better than spilling to disk
- **Consistent:** No GC-related performance variations

Configuration and Setup

Enabling Off-Heap Memory

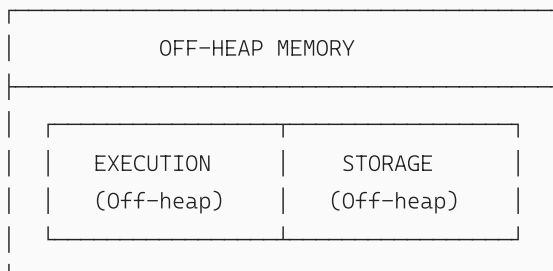
```
// Spark Configuration
spark.conf.set("spark.memory.offHeap.enabled", "true")
spark.conf.set("spark.memory.offHeap.size", "2g")
```

Sizing Guidelines

- **Starting Point:** 10-20% of total executor memory
- **Workload Dependent:** Adjust based on specific requirements
- **Memory Pressure:** Increase if frequent GC pauses observed
- **Monitoring:** Track off-heap usage through Spark UI

Architecture Integration

Off-Heap Memory Structure:



Implementation Considerations

When to Use Off-Heap Memory

- **Large Datasets:** Working with very large cached datasets
- **GC Sensitivity:** Applications experiencing frequent GC pauses
- **Long-running Jobs:** Streaming applications or long-running analytics
- **Memory-intensive Operations:** Complex joins and aggregations

When to Avoid Off-Heap Memory

- **Small Datasets:** Overhead not justified for small workloads
- **Simple Operations:** Basic transformations don't benefit significantly
- **Development Complexity:** Additional complexity not warranted
- **Resource Constraints:** Limited total memory available

Memory Optimization Strategies

Monitoring and Diagnostics

Key Metrics to Track

- **Memory Utilization:** Percentage of used vs. available memory
- **GC Frequency:** Number of GC cycles per time unit
- **GC Duration:** Time spent in garbage collection
- **Spill Metrics:** Amount of data spilled to disk
- **Cache Hit Ratio:** Effectiveness of cached data

Tools and Techniques

1. **Spark UI:** Built-in monitoring interface
2. **Spark History Server:** Historical job analysis
3. **JVM Monitoring:** GC logs and heap dumps
4. **Cluster Monitoring:** Resource utilization across cluster

Tuning Recommendations

Memory Configuration

```
// Recommended starting configurations
spark.executor.memory = "8g"
spark.memory.fraction = "0.6" // Default, adjust if needed
spark.memory.storageFraction = "0.5" // Balance execution/storage
spark.executor.memoryOverhead = "1g" // 10-15% of executor memory
```

Workload-Specific Tuning

- **Compute-Heavy:** Increase execution memory fraction
- **Cache-Heavy:** Increase storage memory fraction
- **Shuffle-Heavy:** Ensure adequate execution memory
- **Streaming:** Consider off-heap for stable performance

Common Performance Issues

Memory-Related Problems

1. **OutOfMemoryError:** Insufficient executor memory
2. **Frequent Spills:** Inadequate memory for operations
3. **Long GC Pauses:** Oversized heaps or memory pressure
4. **Cache Evictions:** Insufficient storage memory

Diagnostic Approaches

1. **Analyze Spark UI:** Identify memory bottlenecks
2. **Review GC Logs:** Understand garbage collection patterns
3. **Monitor Spill Metrics:** Track data spilled to disk
4. **Profile Applications:** Identify memory-intensive operations

Advanced Topics

Memory-Aware Scheduling

Spark's scheduler considers memory availability when:

- Assigning tasks to executors
- Deciding on task parallelism
- Managing resource allocation
- Handling memory pressure

Integration with Cluster Managers

YARN Integration

- Memory requests include executor + overhead
- Container memory limits enforced
- Dynamic allocation based on memory availability

Kubernetes Integration

- Pod memory limits and requests
- Resource quotas and limits
- Horizontal pod autoscaling considerations

Future Developments

Adaptive Query Execution (AQE)

- Dynamic memory allocation based on runtime statistics
- Improved memory utilization through adaptive planning
- Better handling of skewed data distributions

Columnar Memory Formats

- Arrow-based memory representation
- Improved memory efficiency for analytical workloads
- Better compression and encoding

Summary and Key Takeaways

Critical Concepts

1. **Unified Memory:** Dynamic allocation between execution and storage
2. **Priority System:** Execution memory has higher priority
3. **GC Impact:** On-heap memory subject to garbage collection
4. **Off-heap Benefits:** Consistent performance without GC pauses

Best Practices

- Monitor memory usage continuously
- Size executors appropriately for workload
- Use caching judiciously
- Consider off-heap for GC-sensitive applications
- Understand dynamic memory allocation rules

Performance Optimization

- Balance execution and storage memory needs
- Minimize garbage collection impact
- Optimize for specific workload patterns
- Use appropriate storage levels

- Monitor and adjust configurations based on metrics